

HƯỚNG DẪN SINH VIÊN

TẠO AI AUDIT LOG

CSD20x

(RBL Insight Framework - AI Reflection 30%)

I. AI AUDIT LOG LÀ GÌ?

AI Audit Log KHÔNG phải là: Nơi copy-paste toàn bộ chat history với AI

AI Audit Log LÀ: Nhật ký ghi lại NHỮNG QUYẾT ĐỊNH QUAN TRỌNG khi làm việc với AI, chứng minh bạn có tư duy phản biện và tự chủ.

Mục đích:

- Chứng minh bạn hiểu code/giải pháp, không phải chỉ copy từ AI
- Thể hiện khả năng phát hiện lỗi sai của AI (hallucination)
- Chứng minh giá trị con người (Human Delta) mà AI không thể thay thế

II. CORE PROMPT LÀ GÌ?

Nguyên tắc vàng: Chỉ ghi những prompt mà nếu KHÔNG CÓ NÓ, project của bạn sẽ KHÁC VỀ BẢN CHẤT (kiến trúc, thiết kế, cách giải quyết), không phải chỉ khác về syntax hay formatting.

Ba loại Core Prompt (BẮT BUỘC ghi)

Loại Prompt	✓ CORE (Phải ghi)	✗ AUXILIARY (Không ghi)
Decision-Making	"So sánh hiệu suất giữa BST và AVL Tree khi dữ liệu đầu vào đã được sắp xếp."	"Cách khai báo mảng trong C++/Java/Python?"
Problem-Solving	"Làm sao để xóa một nút trong Doubly Linked List mà không làm mất liên kết của các nút còn lại?"	"Sửa lỗi thiếu dấu chấm phẩy (;) ở dòng 15."
Verification	"Hàm xóa nút này của AI có xử lý trường hợp cây rỗng hoặc xóa nút lá (leaf node) không?"	"Tạo hàm khởi tạo (Constructor) cho class Student."

**** Mẹo kiểm tra nhanh:** Nếu không có prompt này → Project vẫn giống 90% → LOẠI.
Nếu không có prompt này → Project khác về bản chất → GHI.

III. SỐ LƯỢNG PROMPTS CẦN GHI

Loại bài	Core Prompts (Min-Max)	Hallucination Detection
Assignment (CSD20x)	8 - 16	≥ 2 cases

**** LƯU Ý:** Mỗi DTC component (Decomposition, Pattern Recognition, Abstraction, Algorithms) phải có ÍT NHẤT 1 core prompt tương ứng.

IV. CÁCH ĐIỀN AI AUDIT LOG

Cấu trúc mỗi entry gồm 7 phần:

1. **Entry #:** Số thứ tự (001, 002, ...)
2. **Prompt Type:** DECISION / PROBLEM-SOLVING / VERIFICATION
3. **Stage/Component:** Decomposition, Pattern Recognition, Abstraction, Algorithms
4. **Problem/Context:** Vấn đề đang gặp (1-2 câu ngắn gọn)
5. **Prompt to AI:** Câu lệnh gửi cho AI (NGUYỄN VĂN)
6. **AI Response (Summary):** Tóm tắt phản hồi của AI (2-4 câu)
7. **Human Delta & Reflection:** PHẦN QUAN TRỌNG NHẤT - Phải trả lời 4 câu hỏi:

4 câu hỏi bắt buộc trong Human Delta & Reflection:

- **Critical Thinking:** AI đúng/sai ở đâu? Phát hiện hallucination?
- **Contextualization:** Bối cảnh thực tế mà AI không biết?
- **Creative Synthesis:** Tôi đã thay đổi/kết hợp/tối ưu như thế nào?
- **Decision Ownership:** Quyết định cuối cùng và lý do?

V. VÍ DỤ CỤ THỂ

✔ Ví dụ TỐT (Entry hoàn chỉnh)

Entry #: 003

Prompt Type: DECISION-MAKING

Stage/Component: Algorithms

Problem/Context: Xây dựng tính năng "Lịch sử cuộc gọi nhỡ" (Missed Calls) trong ứng dụng điện thoại. Cần lưu trữ các số điện thoại mới nhất và cho phép xóa/quay lại dễ dàng.

Prompt to AI: "Tôi nên sử dụng mảng (Array) hay Danh sách liên kết đơn (Singly Linked List) để quản lý danh sách cuộc gọi nhỡ hiệu quả nhất trong Java?"

AI Response (Summary): AI gợi ý sử dụng **Singly Linked List** vì tính linh hoạt khi thêm phần tử mới vào đầu danh sách (addFirst) có độ phức tạp $O(1)$.

Human Delta & Reflection:

- **Critical Thinking:** AI gợi ý đúng về mặt thêm phần tử, nhưng quên rằng người dùng thường có nhu cầu xóa một số điện thoại bất kỳ trong danh sách hoặc quay lại (back) xem số trước đó. Nếu dùng Singly Linked List, việc xóa một nút ở giữa sẽ tốn $O(n)$ để tìm nút trước nó.
- **Contextualization:** Ứng dụng thực tế cần tính năng "Undo/Redo" hoặc "Previous/Next" khi duyệt danh sách cuộc gọi. Singly Linked List chỉ duyệt được 1 chiều, gây bất tiện.
- **Creative Synthesis:** Tôi đã so sánh 2 phương án: (1) Singly Linked List và (2) Doubly Linked List. Kết quả: Doubly Linked List cho phép duyệt hai chiều và xóa một nút cực nhanh nếu đã có con trỏ trỏ tới nó.
- **Decision Ownership:** Quyết định chọn Doubly Linked List. Mặc dù tốn bộ nhớ hơn (do có thêm con trỏ prev), nhưng nó tối ưu hóa trải nghiệm người dùng cho các thao tác duyệt ngược và xóa/chèn tại vị trí bất kỳ.
- **Evidence:** Mã nguồn cài đặt class Node với hai con trỏ next, prev và bảng so sánh thời gian thực thi thao tác deleteNode.

✗ Ví dụ KÉM (Không đạt)

Entry #: 007

Human Delta & Reflection:

"AI đã viết cho tôi hàm duyệt cây Binary Search Tree theo thứ tự In-order. Tôi đã copy vào bài và code chạy rất tốt, không có lỗi gì."

Tại sao KÉM:

- Không trả lời 4 câu hỏi cốt lõi: Không có phân tích về ưu/nhược điểm của giải thuật.
- Thiếu Critical Thinking: Không phản biện được AI (ví dụ: AI dùng đệ quy hay dùng ngăn xếp? Đệ quy có nguy cơ gây tràn bộ nhớ Stack nếu cây quá sâu không?).
- Không có Decision-making process: Không giải thích được tại sao lại chọn duyệt In-order mà không phải Pre-order hay Post-order cho bài toán cụ thể này.
- Không có evidence: Chỉ nói "code chạy tốt" mà không có bằng chứng chạy thử (test cases) hoặc hình ảnh minh họa quá trình trace thuật toán trên giấy/bảng.

VI. PHÁT HIỆN HALLUCINATION (BẮT BUỘC)

Hallucination là gì? Khi AI đưa ra thông tin SAI nhưng trình bày rất tự tin (fake papers, fake APIs, logic errors)

5 loại Hallucination phổ biến:

- **Library/Function Fabrication:** AI tạo ra các phương thức hoặc class không tồn tại trong thư viện chuẩn của ngôn ngữ (ví dụ: `list.sort_descending()` không tồn tại trong Java/C++)
- **Oversimplification (Bỏ qua edgecases):** Đây là lỗi phổ biến nhất. AI cung cấp thuật toán chạy đúng với dữ liệu thông thường nhưng CRASH khi gặp danh sách rỗng, cây chỉ có một nút, hoặc xóa nút gốc.
- **Efficiency Logic Error:** AI khẳng định một thuật toán là "tốt nhất" mà không xét đến bối cảnh. Ví dụ: Khuyến dùng Quick Sort cho dữ liệu đã gần như sắp xếp xong (trong khi Insertion Sort hoặc Bubble Sort cải tiến lại hiệu quả hơn).
- **Complex Logic Fabrication:** AI mô tả các thuật toán phức tạp (như AVL rotation hoặc Dijkstra) nhưng các bước cập nhật con trỏ bị sai lệch, dẫn đến mất dữ liệu.
- **Context Misunderstanding:** AI không hiểu các ràng buộc cụ thể của đề bài (ví dụ: đề yêu cầu dùng Singly Linked List nhưng AI lại dùng ArrayList của thư viện có sẵn).

Ví dụ Hallucination Detection:

- **Tình huống:** SV thực hiện hàm xóa nút lá trong Binary Search Tree (BST).
- **Prompt:** "Viết hàm Java xóa một nút lá (leaf node) trong cây BST."
- **AI Response:** AI cung cấp đoạn code xóa nút bằng cách gán `node = null`; AI khẳng định: *"Đoạn code này sẽ xóa sạch nút khỏi bộ nhớ."*
- **Detection:** Khi chạy thử hoặc trace thuật toán, SV nhận thấy việc gán tham số `node = null` chỉ làm thay đổi biến cục bộ trong hàm, **con trỏ từ nút cha vẫn trỏ vào nút cũ** → **LOGIC ERROR.**
- **Corrective Action:** SV phải tìm nút cha (parent node), xác định nút cần xóa là con trái hay con phải, sau đó cập nhật con trỏ của nút cha thành null.

VII. CHECKLIST TỰ KIỂM TRA (TRƯỚC KHI NỘP)

Kiểm tra MỖI ENTRY phải pass $\geq 4/5$ tiêu chí:

- ☐ Prompt này ảnh hưởng đến quyết định quan trọng trong project?
- ☐ Nếu không có prompt này, project có thay đổi về architecture/design/approach?
- ☐ Tôi có thể giải thích lý do tại sao chọn/không chọn gợi ý của AI?
- ☐ Có minh chứng cụ thể (code, metrics, comparison) cho quyết định này?
- ☐ Prompt này phản ánh tư duy phản biện, không phải chỉ copy AI output?

**** Nếu entry KHÔNG pass $\geq 4/5$ → LOẠI BỎ, không ghi vào Log**

Kiểm tra TỔNG THỂ Log:

- ☐ Số lượng entries nằm trong range (min-max) theo loại bài?
- ☐ Mỗi DTC component có ít nhất 1 core prompt?
- ☐ Đã phát hiện $\geq$ số lượng hallucination yêu cầu?
- ☐ Mỗi entry đều có Human Delta đầy đủ (4 câu hỏi)?

☐ Có evidence cho $\geq 70\%$ entries?

VIII. LƯU Ý QUAN TRỌNG

- **AI Audit Log chiếm 30% tổng điểm - KHÔNG THỂ BỎ QUA**
- Giảng viên sẽ hỏi vấn đáp (Oral Vivas) về Log - Phải trả lời được
- Log giả/kém chất lượng $\rightarrow$ 0 điểm AI Reflection (mất 30%)
- Chất lượng quan trọng hơn số lượng - Đừng ghi nhiều entries trivial
- Nộp đúng deadline - Log là bộ phận **BẮT BUỘC** của bài nộp

Chúc các bạn hoàn thành AI Audit Log thành công!

© 2026 FPT University - Computing Fundamental Department